
Kubernetes Dynamic WebApp Lab — Full Documentation

Prerequisites

- A running Kubernetes cluster (e.g., **kops on AWS**)
- kubectl configured for the cluster
- Docker installed locally
- Docker Hub account
- Optional: Access to AWS console for NodePort rules or LoadBalancer

---

Step 1 — Prepare the Web App Docker Image

1️⃣ Create Project Directory

```
mkdir ~/docker-lab
cd ~/docker-lab
```

2️⃣ Create HTML Template

Create index.html.template:

```
<!DOCTYPE html>
<html>
<head>
  <title>${APP_NAME}</title>
</head>
<body>
  <h1>${WELCOME_MESSAGE}</h1>
  <p>Theme: ${APP_THEME}</p>
  <p>Environment: ${ENVIRONMENT}</p>
  <p>License Key: ${LICENSE_KEY}</p>
</body>
</html>
```

Explanation:

- ${APP_NAME}, ${WELCOME_MESSAGE}, ${APP_THEME}, ${ENVIRONMENT}, ${LICENSE_KEY} are placeholders.
- envsubst will replace these with environment variables at runtime.

3️⃣ Create Dockerfile

```
FROM nginx:alpine

# Install envsubst
RUN apk add --no-cache gettext

# Copy template
COPY index.html.template /usr/share/nginx/html/index.html.template

# Replace template at container start
CMD sh -c "envsubst < /usr/share/nginx/html/index.html.template > /usr/share/nginx/html/index.html && nginx -g 'daemon off;'"
```

Explanation:

- Installs envsubst to replace environment variables.
- Copies template inside container.
- CMD runs envsubst to generate index.html before starting Nginx.

4️⃣ Build and Test Docker Image

```
docker build -t demo-webapp:latest .
docker run -p 8080:80 \
  -e APP_NAME="Docker App" \
  -e WELCOME_MESSAGE="Hello From Docker Class" \
  -e APP_THEME="Dark-Light" \
  -e ENVIRONMENT="Prod-Env" \
  -e LICENSE_KEY="ABC-XYZ" \
  demo-webapp:latest
```

Verify:

curl http://localhost:8080

✅ You should see dynamic HTML with your environment variable values.

5️⃣ Push Image to Docker Hub

docker tag demo-webapp:latest highteaching0124/demo-webapp:latest
docker push highteaching0124/demo-webapp:latest

Step 2 — Create ConfigMap (Non-Sensitive Data)

webapp-config.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: webapp-config
data:
  APP_NAME: "Demo WebApp Dynamic"
  APP_THEME: "dark"
  ENVIRONMENT: "production"
  WELCOME_MESSAGE: "Welcome to Kubernetes Dynamic Lab!"
```

Apply:

kubectl apply -f webapp-config.yaml
kubectl get configmap webapp-config

Step 3 — Create Secret (Sensitive Data)

Encode LICENSE_KEY in Base64

echo -n "PRO-ABC-123-XYZ" | base64
Output: UFJPLUFCRC0tMTIzLVhZWg==

webapp-secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: webapp-secret
type: Opaque
data:
  LICENSE_KEY: UFJPLUFCRC0tMTIzLVhZWg==
```

Apply:

```
kubectl apply -f webapp-secret.yaml
kubectl get secret webapp-secret
```

Step 4 — Create HTML Template ConfigMap

webapp-html.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: webapp-html
data:
  index.html.template: |
    <!DOCTYPE html>
    <html>
    <head>
        <title>${APP_NAME}</title>
    </head>
    <body>
        <h1>${WELCOME_MESSAGE}</h1>
        <p>Theme: ${APP_THEME}</p>
        <p>Environment: ${ENVIRONMENT}</p>
        <p>License Key: ${LICENSE_KEY}</p>
    </body>
    </html>
```

Apply:

kubectl apply -f webapp-html.yaml
kubectl get configmap webapp-html

Step 5 — Create Deployment

webapp-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo-webapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: demo-webapp
  template:
    metadata:
      labels:
        app: demo-webapp
    spec:
      containers:
        - name: web-container
          image: highteaching0124/demo-webapp:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 80
          envFrom:
            - configMapRef:
                name: webapp-config
          volumeMounts:
            - name: secret-volume
              mountPath: "/etc/license"
              readOnly: true
            - name: html-template
              mountPath: /usr/share/nginx/html/index.html.template
              subPath: index.html.template
```

```
        command:
          - sh
          - -c
          - |
            export LICENSE_KEY=$(cat /etc/license/LICENSE_KEY)
            envsubst < /usr/share/nginx/html/index.html.template > /usr/share/nginx/html/index.html
            nginx -g 'daemon off;'
      volumes:
        - name: secret-volume
          secret:
            secretName: webapp-secret
        - name: html-template
          configMap:
            name: webapp-html
```

Apply deployment:

```
kubectl apply -f webapp-deployment.yaml
kubectl rollout restart deployment demo-webapp
kubectl get pods
```

Step 6 — Expose Service

webapp-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: demo-webapp-service
spec:
  selector:
    app: demo-webapp
  type: NodePort
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

Apply:

```
kubectl apply -f webapp-service.yaml
kubectl get svc demo-webapp-service
```

Access:

1. Get Node IPs:

```
kubectl get nodes -o wide
```

2. Combine Node External IP + NodePort (e.g., 30975):

http://<NODE_EXTERNAL_IP>:30975

✅ You should see dynamic HTML page.

Tip: If NodePort is blocked on AWS, adjust the security group.

Step 7 — Test Dynamic Updates

Update ConfigMap

```
kubectl edit configmap webapp-config
```

- Change WELCOME_MESSAGE or APP_THEME.

Restart deployment:

```
kubectl rollout restart deployment demo-webapp
kubectl get pods
kubectl exec -it <pod-name> -- cat /usr/share/nginx/html/index.html
```

Update Secret

```
kubectl edit secret webapp-secret
# Update LICENSE_KEY (base64 encoded)
kubectl rollout restart deployment demo-webapp
```

✅ Your web app now dynamically reflects updates without rebuilding the image.

Step 8 — Using a Private Docker Image

1. Create Docker registry secret:

```
kubectl create secret docker-registry dockerhub-secret \
  --docker-username=highteaching0124 \
  --docker-password=<DOCKER_PASSWORD_OR_TOKEN> \
  --docker-email=<EMAIL>
```

2. Add to Deployment:

```
spec:
  imagePullSecrets:
    - name: dockerhub-secret
```

3. Apply deployment:

```
kubectl apply -f webapp-deployment.yaml
kubectl rollout restart deployment demo-webapp
```

✅ Kubernetes can now pull private Docker images.

Key Concepts Learned

Concept	Explanation
Docker template	Static image with placeholders
envsubst	Replaces environment variables at runtime
ConfigMap	Non-sensitive dynamic config in Kubernetes

Secret	Sensitive config stored securely
Volume Mount	Makes ConfigMap or Secret files accessible to pod
Deployment	Ensures desired pod replicas and rolling updates
NodePort / Service	Exposes pods to the outside world
Rolling Update	Allows updating pods without downtime
Private Image	Requires docker-registry secret to pull

Best Practices

- Use **ConfigMaps** for environment-specific configs.
- Use **Secrets** for passwords or license keys.
- Always **restart pods** after changing ConfigMap or Secret.
- Prefer **versioned Docker images** instead of :latest.
- Consider **LoadBalancer service** for cloud environments instead of NodePort.

✅ **Lab Flow Summary**

1. Build Docker image with template.
2. Test locally with environment variables.
3. Push image to Docker Hub.
4. Create ConfigMap and Secret in Kubernetes.
5. Mount template, ConfigMap, and Secret in Deployment.
6. Run envsubst inside container to generate final HTML.
7. Expose via NodePort / Service.
8. Test dynamic updates by editing ConfigMap or Secret.
9. Support private Docker images using imagePullSecrets.